SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and

Big Data Analytics **COURSE CODE: CSA15**

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4)

Question Count: 5 **Total Marks: 50**

Code of Conduct

I S. Nitheesh(192524329) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Nitheesh

1. HDFS-based storage solution for a media platform

Constraints:

- Data availability > 99.9%
- Replication factor ≤ 3
- Cost-efficient storage growth

Solution:
Use an HDFS cluster with a **replication factor of 3** for important media data. Store files as large HDFS blocks and distribute the blocks across different DataNodes and racks.

Design:

- **NameNode:** Maintains metadata and file information.
- **DataNodes:** Store actual media blocks.
- **Replication factor = 3:** Each block has three copies on different DataNodes, preferably across different racks.
- **Rack awareness:** Prevents all replicas from being placed in the same rack.
- **Compression:** Compress metadata, logs, thumbnails, and suitable media formats to reduce storage requirements.
- **Storage tiers:** Frequently accessed media can use faster storage, while older media can be moved to cheaper storage.
- **Monitoring:** Monitor DataNode failures and automatically re-replicate missing blocks.

Justification:
Replication factor 3 provides high availability and fault tolerance while staying within the constraint. Rack-aware placement protects data even if an entire rack fails. Compression and storage tiering reduce storage costs as the media collection grows.

---

2. MapReduce workflow for log processing within 20 minutes

Constraints:

- Processing must finish within 20 minutes
- Limited number of cluster nodes

Solution:
Design a MapReduce job with **parallel mappers and optimized reducers**.

Workflow:

Log Files → Input Splits → Mapper → Shuffle & Sort → Reducer → Output

Mapper:

- Each mapper processes a separate input split.
- Extract required information such as IP address, timestamp, URL, error code, or response time.
- Convert the data into key-value pairs.

Example:

IP_Address → 1

Reducer:

- Receives grouped keys from mappers.
- Calculates counts, averages, errors, or other required statistics.

Task distribution:

- Divide the log files into multiple input splits.
- Assign one mapper to each split.
- Use the available nodes efficiently by running multiple mapper tasks per node, subject to CPU and memory limits.
- Use a suitable number of reducers to avoid reducer bottlenecks.
- Use **combiners** where possible to reduce data transferred during shuffle.
- Compress intermediate/map output to reduce network traffic.

Justification:
Parallel processing reduces execution time. Combiners reduce shuffle traffic, while appropriate partitioning prevents one reducer from becoming overloaded. Monitoring task progress allows resources to be reallocated to slow tasks. These optimizations help the job meet the **20-minute batch window** despite limited nodes.

---

3. YARN scheduler policy for ETL, reporting, and ML

Constraints:

- Fair resource sharing
- ETL must receive higher priority
- Recovery from node failure **Proposed scheduler: Capacity Scheduler**

Queue design:

Queue	Priority	Purpose	Suggested Capacity
ETL	High	Data ingestion and transformation	50%
Reporting	Medium	Reports and dashboards	25%
Machine Learning	Normal	Training and analysis	25%

Policy:

1. Give **ETL jobs high priority** because they are important for downstream processing.
2. Reserve a minimum capacity for reporting and ML so that ETL does not completely starve them.
3. Use queue limits to prevent a single application from consuming all resources.
4. Allow unused resources to be temporarily used by other queues.
5. Enable YARN application recovery so applications can restart after failures.
6. If a NodeManager fails, YARN detects the failed node and reschedules affected tasks on healthy nodes.

Justification:
The policy gives ETL priority while maintaining fairness for reporting and ML. Dynamic resource allocation improves utilization, and automatic task/application recovery provides fault tolerance.

---

4. Enterprise data ingestion from SAN, NAS, and databases

Constraints:

- Reliable backup
- Minimal duplication
- Multiple data sources

Architecture:

SAN / NAS / Operational DBs
↓
Apache NiFi / Ingestion Layer

↓
Validation + Deduplication
↓
HDFS / Data Lake

↓
Backup / Disaster Recovery
↓
Analytics / Reporting / ML

Components:

1. SAN:
Use connectors or agents to ingest enterprise files and storage data into the ingestion layer.

2. NAS:
Monitor NAS directories and transfer newly created or modified files.

3. Operational databases:
Use JDBC-based ingestion or CDC (Change Data Capture) to capture new and changed records instead of repeatedly copying complete tables.

4. Apache NiFi:
NiFi manages data flow, routing, transformation, monitoring, and error handling.

5. Deduplication:
Generate a unique file/record identifier using metadata or hashing. Before storing data, check whether the identifier already exists.

6. HDFS/Data Lake:
Store the centralized data using replication for fault tolerance.

7. Backup:
Maintain backup copies in a separate storage system or disaster-recovery environment.

Justification:
NiFi provides centralized ingestion and monitoring. CDC minimizes unnecessary data transfer, while hashing/metadata-based deduplication reduces duplicate storage. HDFS replication and separate backups improve reliability.

---

5. End-to-end Hadoop ecosystem solution

Constraints:

- Secure data movement
- High availability
- Integration with Apache NiFi/ETL

Architecture:

Data Sources

↓
Apache NiFi
↓
Secure Data Transfer
↓
HDFS
↓
YARN
↓
MapReduce / Spark / Hive
↓
Analytics / ML / Reports

Components and design:

1. Apache NiFi – Data ingestion

- Collect data from databases, files, APIs, SAN/NAS, and applications.
- Route and transform data before storing it.
- Use back-pressure and queues to handle traffic spikes.

2. Secure data movement

- Use **TLS/SSL** for data transmission.
- Use **Kerberos** for Hadoop authentication.
- Apply access control using Hadoop permissions and, where required, Apache Ranger.
- Encrypt sensitive data at rest.

3. HDFS – Storage

- Use replication factor **3** for important data.
- Use rack awareness to protect against rack failures.
- Use an HA NameNode configuration with **Active and Standby NameNodes**.

4. YARN – Resource management

- Allocate cluster resources between ETL, analytics, reporting, and ML workloads.
- Use scheduler priorities and queue limits.

5. Processing

- **MapReduce:** Batch processing.

- **Spark:** Fast analytics and machine learning.
- **Hive:** SQL-based analysis and reporting.

6. High availability

- Active/Standby NameNodes
- Multiple DataNodes
- Replicated HDFS blocks
- YARN ResourceManager HA
- Automatic task/application recovery

Justification:
NiFi provides controlled and monitored data ingestion, while TLS and Kerberos secure data movement and authentication. HDFS replication and NameNode HA provide high availability. YARN manages computing resources, and Hive, Spark, and MapReduce support different analytical workloads. Therefore, the architecture satisfies **security, high availability, scalability, and ETL integration constraints**.